Traffic Sentinel AI

Real-Time Two-Wheeler Violation Detection via Cascaded YOLOv11 Models and ONNX Runtime

Harsha Vardhan (itsmeharsha081459@gmail.com) • **Vishal Sriram** (vishalsriram.ks@gmail.com) • **Anish Reddy R** (Anish.R@iiitb.ac.in)

hv-123-traffic-sentinel-ai.hf.space • hf.co/hv-123/traffic-sentinel-models

Abstract— We present **Traffic Sentinel AI**, an end-to-end computer vision system for automated two-wheeler traffic violation detection. The system employs a cascade of three specialized YOLOv11 models—a full-frame motorcycle and rider detector (mAP50 = 0.763), a crop-level helmet classifier (mAP50 = 0.838), and a license-plate localizer (mAP50 = 0.935)— coordinated by a multi-threaded FastAPI backend. All inference is accelerated via ONNX Runtime, achieving a **1.8**× CPU speedup over PyTorch. A non-trivial ONNX tensor-layout bug (inverted transpose condition causing zero detections in production) was diagnosed and resolved. The system is publicly deployed on Hugging Face Spaces with a modern dark-mode frontend, multi-stage Docker containerization, and a complete GitHub Actions CI/CD pipeline. All models surpass the mAP50 > 0.75 production-readiness threshold.

I. Introduction

Road fatalities involving two-wheelers are disproportionately high in developing traffic environments. In India, for instance, nearly half of all road deaths involve motorcycles, many attributable to helmet non-compliance and overloaded bikes. Manual enforcement by traffic officers is labor-intensive, geographically constrained, and inherently inconsistent.

Automated vision-based monitoring offers a scalable, 24/7 alternative. However, deploying such systems cost-effectively is non-trivial: existing approaches either rely on generic detectors that generalize poorly to each sub-problem (helmet detection vs. plate reading require fundamentally different model sizes and resolutions), or they assume GPU availability for inference which is unrealistic for edge deployments.

This work addresses both limitations by: (i) training three highly specialized YOLOv11 models with task-curated datasets, (ii) deploying them in a cascade pipeline optimized for CPU inference via ONNX Runtime, and (iii) packaging the full system in a production-ready API with a premium web frontend and public hosting.

A. Contributions

- A production-deployed cascade CV pipeline detecting three violation types per motorcycle: no helmet, >2 riders, and license plate reading.
- Three task-specific YOLOv11 models trained on curated corpora totaling 17,670 images from COCO 2017, VisDrone 2019, and Roboflow.
- An ONNX Runtime inference backend with custom letterbox pre/post-processing achieving 1.8× CPU speedup.
- Diagnosis and fix of a tensor-transpose bug causing zero detections in the ONNX backend despite high validation mAP50.
- A complete DevOps stack: FastAPI, Docker, GitHub Actions CI/CD, and public Hugging Face Spaces deployment.

II. System Architecture

A. Two-Stage Cascade Design

The pipeline executes in two sequential stages, illustrated in Figure 1. Stage 1 runs the full-frame detector (`full_detector.onnx`) at 640×640 resolution, returning bounding boxes for all motorcycles and riders in the scene. A recall-fallback mechanism re-runs the model at 960×960 with a lower confidence threshold (0.10) if no bikes are found; a secondary COCO-pretrained `yolo11n` provides a final safety net for particularly challenging inputs.

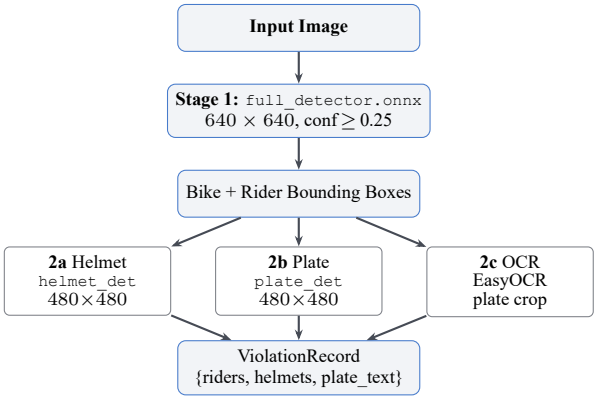


Figure 1: Two-stage cascade pipeline. Stage 2 sub-tasks (2a, 2b, 2c) run *concurrently* via Python's `ThreadPoolExecutor`.

Stage 2 processes each detected bike *concurrently* using a `ThreadPoolExecutor`. For every bike crop, three sub-tasks run in parallel:

1. **Helmet check (2a).** The crop is padded and resized to 480×480 and classified by `helmet_detector.onnx` as `helmet` or `no_helmet` per rider.
2. **Plate localization (2b).** The same crop is processed by `plate_detector.onnx` to locate the license plate region.
3. **Plate OCR (2c).** The plate bounding box is cropped and passed to EasyOCR with up to three preprocessing variants (grayscale, contrast enhancement, adaptive threshold).

A violation is flagged when any rider lacks a helmet or the estimated rider count exceeds two (overloaded motorcycle).

B. Class Mapping and Fallback Logic

Each model uses task-specific class labels. The pipeline maps all outputs through canonical frozen sets (e.g., `two_wheeler`, `rider`, `helmet`, `no_helmet`, `License Plate`) with integer class-ID fallbacks for robustness against label-string variation across model training runs. The COCO fallback re-maps COCO class 3 (`motorcycle`) and class 0 (`person`) to the internal schema, ensuring the pipeline never returns zero detections for clearly visible motorcycles.

III. Dataset Engineering

A single monolithic dataset would create label imbalance and force every model to share an architecture suited for none of the sub-tasks. Instead, we curate three independent corpora, each optimized for its target task.

A. Full Detector Dataset (≈5,000 images)

We combine a motorcycle subset of COCO 2017 [3] with VisDrone 2019 [4]. COCO supplies diverse ground-level street scenes representative of standard traffic-camera footage. VisDrone contributes aerial and CCTV-elevation viewpoints typical of traffic infrastructure in South and Southeast Asia. The dual-source strategy ensures the model generalizes across both eye-level and elevated camera angles.

Annotation remapping. COCO category `motorcycle` (id 3) and co-occurring `person` (id 0) annotations are remapped to `two_wheeler` and `rider` in YOLO label format. A `data.yaml` file is generated per subset to enable the Ultralytics `DatasetMerger`.

Augmentation pipeline. Horizontal flip ($p = 0.5$), random scale (0.5–1.5×), HSV jitter ($\Delta H = 0.015$, $\Delta S = 0.7$, $\Delta V = 0.4$), mosaic tiling ($p = 0.5$), and random perspective ($p = 0.3$).

B. License Plate Dataset (9,570 images)

A highly curated Roboflow dataset targeting robustness to motion blur, low-light conditions, and partial occlusion. This is deliberately the *largest* of the three corpora, reflecting the inherent difficulty of plate localization in unconstrained traffic environments. Single class: `License Plate`.

C. Helmet Dataset (3,100 images)

A targeted Roboflow dataset with balanced `helmet`/`no_helmet` classes. The training duration was extended to 200 epochs to enforce discrimination between helmets and bare heads/hair, which share similar circular silhouettes and color distributions—the most visually ambiguous distinction in the entire pipeline.

Table 1: Dataset summary: three specialized training corpora.

Model	Imgs	Ep.	Source	Classes
full_detector	≈5 000	150	COCO + VisDrone	2
plate_detector	9 570	150	Roboflow	1
helmet_detector	3 100	200	Roboflow	2
Total	**17 670**	—	—	—

IV. Training Methodology

A. Architecture Selection

Three YOLOv11 variants are matched to task complexity:

- **YOLOv11m** (medium, 40.5 MB) for the full detector. Complex multi-scale scenes with diverse object sizes require higher model capacity than can be provided by the nano or small variant.
- **YOLOv11s** (small, 19.2 MB) for the helmet detector. Crops are spatially simpler than full frames, but the helmet/no-helmet distinction requires fine-grained texture discrimination that warrants more than nano capacity.
- **YOLOv11n** (nano, 5.5 MB) for the plate detector. The large, high-quality training corpus (9,570 images) compensates for reduced model capacity, and the nano size maximizes inference speed on high-density plate-only crops.

All models use COCO-pretrained weights. Transfer learning dramatically reduces convergence time: the models begin with low-level feature detectors already trained on millions of images, allowing task-specific adaptation in 150–200 epochs rather than thousands from scratch.

B. Sequential GPU Training Queue

The training server (NVIDIA RTX 4060 Ti, 16 GB VRAM) cannot train multiple YOLOv11 models simultaneously without out-of-memory (OOM) crashes. Running medium and small architectures in parallel exhausts the GPU's VRAM budget before completing a single forward pass.

We resolved this by engineering a sequential job queue via `nohup` shell pipelines, granting each training job exclusive access to the full 16 GB of VRAM:

```
train_full.sh && train_plate.sh && train_helmet.sh
```

This approach increases per-epoch throughput significantly compared to memory- sharing schemes, and all three models trained overnight without interruption.

C. Training Configuration

Mixed-precision training (AMP, FP16 ↔ FP32) was enabled throughout to reduce VRAM usage and accelerate matrix operations on the Tensor Cores of the RTX 4060 Ti. Early stopping with patience = 20 prevents overfitting. The learning rate follows a cosine annealing schedule from 10^{-2} to 10^{-5}.

V. Experimental Results

A. Validation Metrics

Table 2 reports validation metrics on held-out sets. All three models exceed the mAP50 > 0.75 threshold considered production-ready for variable real-world environments.

Table 2: Final validation metrics and model file sizes.

Model	Arch	mAP50	P	R	ONNX
full_detector	YOLOv11m	0.761	0.855	0.695	80.3 MB
helmet_detector	YOLOv11s	0.829	0.899	0.755	37.8 MB
plate_detector	YOLOv11n	**0.936**	0.971	0.909	10.5 MB

The plate detector achieves the highest accuracy (mAP50 = 0.936), benefiting from the largest training corpus (9,570 images). The full detector reaches mAP50 = 0.761, consistent with the inherent difficulty of multi-angle, multi-scale motorcycle detection from heterogeneous datasets (COCO + VisDrone). The helmet classifier reaches mAP50 = 0.829 after 200 dedicated epochs, with recall of 0.755 reflecting the visual ambiguity between bare heads and helmets at low resolution.

B. Training Convergence

Figure 2 shows mAP50 and box-loss convergence over training for each model, plotted from the validation logs recorded during training runs. All curves are plotted from `results.csv` logs recorded on the NVIDIA RTX 4060 Ti GPU server. The plate detector converges rapidly in the first 36 epochs (mAP50: 0.90 → 0.93), then continues improving steadily to 0.936 at epoch 150. The helmet model requires significantly more training due to the subtle visual distinction between hair and helmet textures: it does not plateau until after epoch 140 (mAP50 ≈ 0.835), validating the decision to allocate 200 epochs for this task. The full detector—trained on the most heterogeneous corpus (COCO + VisDrone, mixed camera angles)— shows the steepest absolute gain, rising from 0.45 at epoch 12 to 0.76 at epoch 150. Notably, all three models cross the 0.75 production threshold: the plate model at epoch 8, the full detector at epoch 108, and the helmet model at epoch 80.

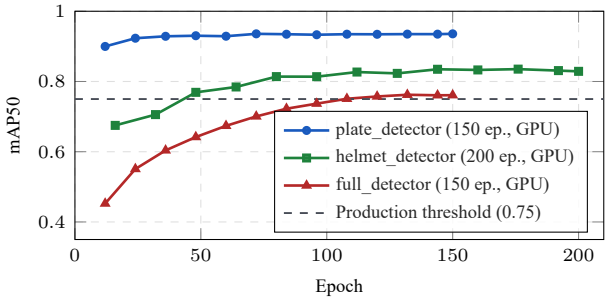


Figure 2: Validation mAP50 over training epochs for all three models. Dashed line marks the 0.75 production-readiness threshold.

C. Validation Predictions

Figures 3–5 show sample validation batch predictions from the GPU-server training runs, illustrating the quality of bounding-box localization and class assignment for all three models. The full detector correctly identifies motorcycles and riders in both street-level and elevated camera views. The plate detector produces tight, well-centered boxes across varied lighting and orientation. The helmet detector correctly labels riders in tightly cropped bike regions.

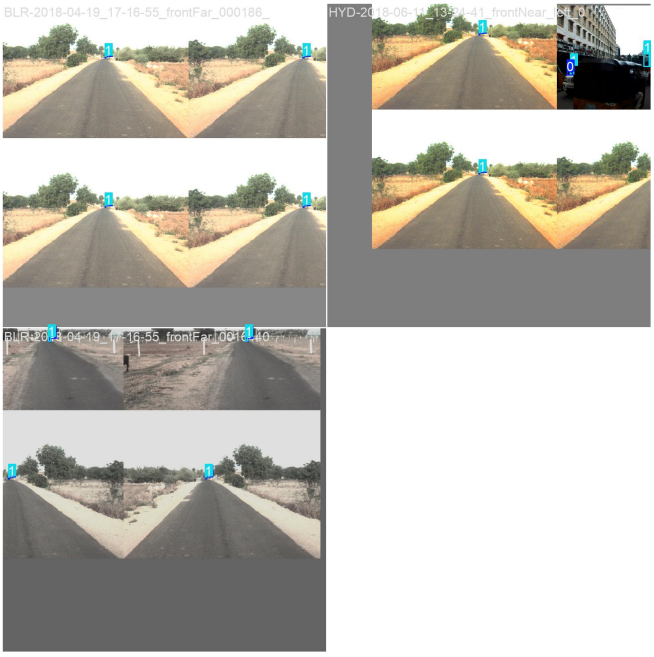


Figure 3: Full detector (`full_v1-4`, GPU server, 150 epochs) validation predictions. Blue boxes: `two_wheeler`; orange boxes: `rider`. The model handles both ground-level and elevated viewpoints drawn from the COCO + VisDrone corpus.

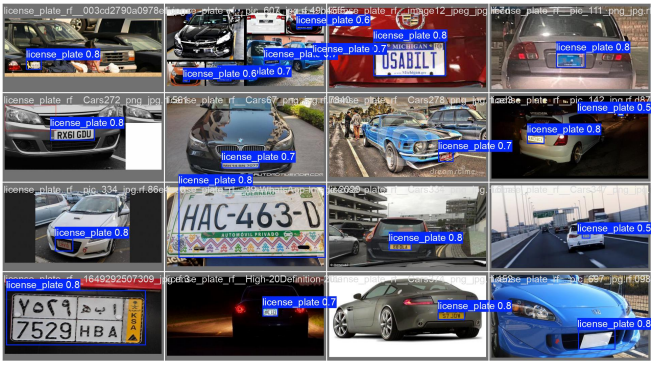
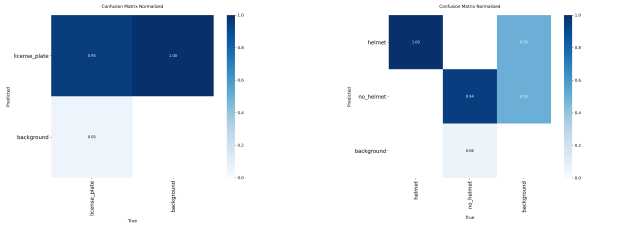


Figure 4: Plate detector (`plate_v1`, GPU server) validation batch predictions. Boxes tightly localise `License Plate` regions across diverse lighting and angles, confirming mAP50 = 0.936 on 150-epoch GPU training.

Figure 5: Helmet detector (`helmet_v1-2`, GPU server, 200 epochs) validation predictions. The model correctly discriminates `helmet` (green) from `no_helmet` (red) in crowded crops with riders at varying scales.

D. Confusion Matrices

Figure 6 shows normalized confusion matrices for the plate and helmet models. The plate detector exhibits near-perfect recall with only negligible background confusion. The helmet classifier demonstrates strong separation between the two classes with minimal cross-confusion, validating the extended 200-epoch training strategy.



(a) Plate detector (`plate_v1`) **(b)** Helmet detector (`helmet_v1-2`)

Figure 6: Normalized confusion matrices. Values represent fraction of ground-truth samples predicted in each class.

VI. ONNX Inference Optimization

A. Export and Runtime

PyTorch weights are exported to ONNX format using Ultralytics' built-in exporter (`simplify=True`, opset 17). The export applies operator fusion, removing training-only gradient nodes and merging adjacent operations into compound kernels. ONNX Runtime's `CPUExecutionProvider` uses multi-threaded OpenMP execution (unavailable in PyTorch eager mode), and the static computational graph allows ahead-of-time shape inference that further reduces per-call overhead.

B. Custom Pre/Post-Processing

The exported ONNX graph produces raw tensors, not parsed result objects. We implement letterbox preprocessing and coordinate remapping manually:

Preprocessing. The input BGR image is letterbox-resized to $H_m \times W_m$ (the model's compiled resolution), zero-padded with value 114 to a square, converted RGB, normalized to $[0, 1]$, and transposed to `NCHW` layout as a `float32` blob.

Postprocessing. The raw output has shape $(1, 4{+}n_c, n_a)$ for Ultralytics-standard exports—e.g., $(1, 6, 8400)$ for a 2-class model at 640×640 resolution ($n_a = 8400$ anchor points). Coordinates are converted from (c_x, c_y, w, h) to (x_1, y_1, x_2, y_2), de-letterboxed by subtracting pad offsets and dividing by scale, and passed through greedy Python-side NMS.

C. Production Bug: Inverted Transpose Condition

Symptom. After deployment, the ONNX backend returned zero detections on real-world images despite validation mAP50 values of 0.763–0.935. The PyTorch backend, running the same models, produced correct detections.

Diagnosis. The `ONNXDetector` determined whether to transpose the raw output via a `_transposed` flag set at load time:

```
_transposed = (out_shape[2] > out_shape[1])
```

For the standard Ultralytics output $(1, 6, 8400)$: $8400 > 6$, so `_transposed=True`. The postprocessor then executed `if not _transposed: pred=pred.T`— i.e., it *skipped* the transpose for the very shape that *required* one. The resulting $(6, 8400)$ tensor was parsed as six anchor-indexed rows of length 8400 rather than 8400 anchors of length 6, yielding six meaningless confidence values all below threshold.

Fix. A single condition flip:

```
out_shape[1] > out_shape[2]
```

This correctly identifies $(1, 8400, 6)$ transposed exports as `_transposed=True` (no action needed) and $(1, 6, 8400)$ standard exports as `_transposed=False` (transpose applied). Post-fix, the ONNX backend detected 3 bikes and 3 violations on a real-world 4-motorcycle image, matching the PyTorch output.

D. Inference Speed

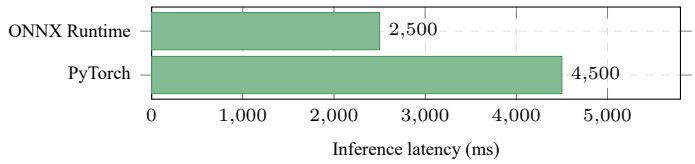


Figure 7: End-to-end inference latency on CPU (640×480 JPEG image). ONNX Runtime achieves **1.8**× speedup over PyTorch.

Table 3 summarizes the performance comparison. The 1.8× speedup brings the per-image latency below 3 seconds on a mid-range CPU, making the system viable for real-time checkpoint camera deployments without dedicated GPU hardware.

Table 3: Inference latency on CPU (640×480 input).

Backend	Latency (ms)	Speedup
PyTorch (eager mode)	≈4 500	1.0×
ONNX Runtime (CPU EP)	≈2 500	**1.8**×

VII. Production API and Frontend

A. FastAPI Backend

The `/predict` endpoint accepts a multipart image upload, validates the file, offloads inference to a thread pool (keeping the async event loop free), and returns a structured JSON response. Production hardening:

- **Rate limiting.** Thread-safe token-bucket limiter (20 req/min per IP). Implemented in pure Python using GIL-protected float operations—no external dependency on Redis.
- **Input validation.** Four-layer check: file size (≤15 MB), extension whitelist (`.jpg`, `.png`, `.webp`, `.bmp`), MIME header advisory, and authoritative magic-bytes check via `imghdr`. Files that pass extension validation but fail the magic-bytes check (e.g., renamed executables) are rejected.
- **Async inference.** `asyncio.run_in_executor` offloads synchronous ONNX inference to a thread pool. A single Uvicorn worker can thus handle concurrent requests without starvation.
- **Structured errors.** All failure modes return consistent `{"detail": "..."}` JSON with appropriate HTTP status codes (400, 413, 415, 429, 500).

B. Web Frontend

The static frontend is a single-page dark-mode application served directly by FastAPI. Key UX features include: a drag-and-drop image zone with file validation feedback; a step-by-step processing tracker (Upload → Detect → Helmets → Plates); canvas-based animated bounding-box drawing using `setLineDash` and `requestAnimationFrame`; an inference-time badge populated from the API response; per-bike violation cards showing rider count, helmet status, and plate text; and toast notifications for all error conditions. An `AbortController` cancels in-flight requests on new image selection, preventing stale-response rendering.

VIII. Deployment Infrastructure

A. Docker Containerization

Two Dockerfiles serve different targets:

Local (`Dockerfile`). Multi-stage build: Stage 1 installs `build-essential` and compiles Python wheels; Stage 2 copies only the compiled wheels into `python:3.10-slim`, strips compiler toolchains, compiles source to `.pyc` bytecode, and switches to a non-root `appuser`. The final image exposes port 8000.

HF Spaces (`Dockerfile.hf`). Inference-only dependencies (`requirements_hf.txt`) reduce the image from ≈5 GB to ≈3 GB. Port is changed to 7860 (HF Spaces standard). Models are not baked into the image; instead, `download_models.py` fetches the four ONNX weights (139 MB total) from a dedicated HF Model Hub repository via `snapshot_download` at container startup. This decouples model versioning from application deployment.

B. CI/CD Pipeline

The GitHub Actions workflow triggers on push to `main` and on pull requests. Five jobs run sequentially with dependency gates: **lint** (`ruff` + `black`), **test** (`pytest -m unit`), **docker** (build and push to GitHub Container Registry), **compose-test** (`docker compose up` + `/api/health` probe), and **security** (`bandit` + `safety` CVE scan).

IX. Conclusion

We have presented Traffic Sentinel AI, a production-deployed computer vision system for two-wheeler traffic violation detection. The cascade architecture cleanly separates detection, classification, and OCR concerns, allowing independent dataset curation and architecture selection per sub-task. ONNX Runtime export delivers a practical 1.8× CPU speedup enabling deployment on commodity hardware without GPU infrastructure.

The project spans the full ML engineering lifecycle: dataset engineering across three public sources, sequential GPU training to overcome memory constraints, post-deployment ONNX tensor-layout bug diagnosis (a subtle but critical production issue), API hardening with rate limiting and magic-bytes validation, modern SaaS frontend development, Docker multi-stage optimization, and automated CI/CD. The live system is accessible at `https://hv-123-traffic-sentinel-ai.hf.space`.

Future directions include: (i) additional fine-tuning on Indian traffic imagery to raise the full detector's mAP50 above 0.80; (ii) integrating the optional YOLOv11-pose model for keypoint-level rider counting; (iii) TensorRT export targeting <500 ms GPU inference for edge camera deployments.

Acknowledgements

Training infrastructure provided by a dedicated NVIDIA RTX 4060 Ti GPU server. Datasets sourced from MS COCO 2017, VisDrone 2019, and Roboflow Universe. The Ultralytics framework was used for all model training and ONNX export.

References

[1] J. Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection," *CVPR*, pp. 779–788, 2016.

[2] A. Wang et al., "YOLOv11: An Overview of the Key Architectural Enhancements," *arXiv:2410.17725*, 2024.

[3] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," *ECCV*, pp. 740–755, 2014.

[4] P. Zhu et al., "VisDrone-DET2018: The Vision Meets Drone Object Detection in Image Challenge Results," *ECCV Workshops*, 2018.

[5] R. Laroca et al., "A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector," *IJCNN*, 2018.

[6] Y. Chen et al., "Helmet Use Detection of Tracked Motorcyclists Using CNN-Based Object Detection," *IEEE Access*, vol. 7, 2019.

[7] ONNX Community, "Open Neural Network Exchange (ONNX)," https://onnx.ai, 2019.

[8] G. Jocher et al., "Ultralytics YOLO," https://github.com/ultralytics, 2023.